Methodology

This chapter records **what** was built, **how** it was built, and **why** each choice was made for the *Read & Access — Tech Report Assistant* project. It is deliberately written at the level of *process and rationale*; the line-level implementation detail belongs to the Implementation chapter, and the running output belongs to the Evaluation chapter.

---

1. Research and Project Approach

The project followed an **iterative, design-led software-engineering methodology**, organised into five repeating activities:

1. **Requirements gathering** — clarifying what the product needed to do for a reader of a technical PDF.
2. **System and interaction design** — sketching the architecture and the user flow.
3. **Incremental implementation** — building one tool/feature at a time behind a small set of shared abstractions.
4. **Continuous evaluation** — manual and peer testing, smoke builds, network inspection, error-injection.
5. **Refinement** — hardening against the failure modes surfaced by step 4 (timeouts, oversized uploads, missing API keys, accessibility gaps, stale uploads).

This is a pragmatic blend of *agile / evolutionary prototyping* (Sommerville, *Software Engineering*) and *user-centred design*: the system was deliberately shipped early in a working state and then improved in tight loops driven by real usage feedback (including a peer attempting to upload a PDF over a different network, which surfaced the upload-timeout class of bugs).

A linear waterfall approach was rejected because the requirements were expected to (and did) change as soon as the first end-to-end version was put in front of users.

---

2. Requirements

2.1 Functional requirements

The functional requirements were derived from the project brief and refined through informal feedback while iterating:

#	Requirement
F1	Upload a technical PDF (≤ 50 MB) and view it in-browser.
F2	Extract the text of the PDF and display it as a searchable, selectable view.
F3	Summarise the whole document or a highlighted passage.
F4	Explain a highlighted passage in plain language.
F5	Define a highlighted term and add it to a personal glossary.

#	Requirement
F6	Locate a claim in the document (Evidence Tracker) and jump to it.
F7	Detect equations, render LaTeX, and explain them.
F8	Convert between engineering units (length, velocity, pressure, force, temperature) with category validation.
F9	Compute International Standard Atmosphere properties at altitude.
F10	Ask free-form questions about the loaded report.

2.2 Non-functional requirements

#	Requirement
N1	One-command reproducible setup (`docker compose up --build`).
N2	Survive uploads of multi-megabyte PDFs (≤ 5-minute end-to-end budget).
N3	Continue to work after page reloads (session persistence).
N4	Provide cancellable, debuggable error states (no silent failures).
N5	Be keyboard-navigable and meet baseline accessibility expectations.
N6	Never commit secrets (OpenAI API key) to the repository.
N7	Auto-clean stored uploads to bound disk usage.

Each later design choice in the methodology is traceable to one of these requirements.

---

3. System Architecture

A **three-tier client–server architecture** was adopted:

```
┌──────────────────────────┐
│   Browser (React SPA)    │
│  components, utils, KaTeX│
└──────────────────────────┘
           │ HTTPS / HTTP
           ▼
┌──────────────────────────┐
│   Nginx reverse proxy    │
│  /  → frontend           │
│  /api/ → backend         │
└──────────────────────────┘
           │
           ▼
┌──────────────────────────┐
│  FastAPI backend (Py)    │
│  routes/ services/ utils │
```

```
                    │                    │
                    ▼                    ▼
                ┌──────────┐      ┌──────────────────┐
                │ uploads/ │      │  OpenAI API      │
                │ (PDFs)   │      │  (chat models)   │
                └──────────┘      └──────────────────┘
```

Justification:

- **Separation of concerns.** UI rendering, business orchestration, and AI inference are all owned by different components, so each can evolve independently (e.g. swapping the LLM provider does not touch the UI).
- **Single origin in production.** Nginx terminates one origin and routes `/` to the SPA and `/api/` to FastAPI. This avoids CORS, hides the backend port, and centralises upload limits and timeouts (`client_max_body_size`, `proxy_read_timeout`).
- **Stateless backend.** All per-user state lives either on disk (`uploads/<id>.pdf`) or in the user's browser (`localStorage`). This simplifies horizontal scaling and removes the need for an authenticated session layer in the MVP.
- **Out-of-process NLP.** The OpenAI API is the natural-language reasoning provider. This was chosen over self-hosting a model because (a) the project is designed to run on student-grade hardware, and (b) the model is the most rapidly improving part of the stack — being able to swap models via an environment variable (`OPENAI_MODEL`) future-proofs the system.

---

4. Technology Choices

For each technology the format is **what + why + alternatives considered**.

4.1 Frontend

- **React 18 + TypeScript.** Component reuse, strong typing for the data contracts that flow between the backend and the UI, large ecosystem. *Alternatives rejected:* plain JavaScript (loses type safety on a project with non-trivial state); Vue/Svelte (smaller community for the specific PDF/KaTeX libraries needed).
- **`react-pdf` / `pdf.js`.** In-browser PDF rendering with the same engine Mozilla ships in Firefox. Avoids a server-side rendering step.
- **KaTeX.** Fast, synchronous LaTeX rendering. Chosen over MathJax because the synchronous, deterministic output is easier to embed inside a React component tree.
- **Axios.** Predictable error shape (`AxiosError`), built-in cancellation via `AbortController`, upload-progress events.

4.2 Backend

- **FastAPI + Uvicorn.** Asynchronous I/O (important when fan-out to OpenAI is the main bottleneck), Pydantic models for request validation, auto-generated OpenAPI documentation at `/docs`. *Alternatives rejected:* Flask (synchronous by default, no native validation); Django (too heavyweight for an API-only service).

- **PyPDF2.** Pure-Python text extraction with no system dependencies. Good enough for native-text PDFs, which is the dominant case for engineering reports. *Alternative considered:* `pdfplumber` and `pdfminer.six` — more accurate on complex layouts but slower and add complexity. *Acknowledged limitation:* scanned image-only PDFs are not supported and would require OCR (e.g. Tesseract). This is documented as future work.
- **OpenAI Python SDK.** Official client, model name is configurable via the `OPENAI_MODEL` env var (default `gpt-3.5-turbo`).

4.3 Infrastructure and deployment

- **Docker + Docker Compose.** Single-command setup, identical environment for the developer and any peer who clones the repo. Mirrors the production pattern and removes "works on my machine" failures.
- **Nginx reverse proxy.** Single origin, request size limits, timeout budgets, and graceful upstream-failure handling (`resolver_timeout`, `proxy_intercept_errors`, `error_page`).

4.4 Persistence

- **Filesystem for PDFs** (`uploads/<id>.pdf`) — trivially simple and survives process restarts. A background `asyncio` task prunes files older than `UPLOAD_RETENTION_HOURS` (default 24 h).
- **In-memory `reports_store` dict** for parsed report metadata. Acknowledged trade-off: state is lost on backend restart; persisting to a database is listed as future work.
- **Browser `localStorage`** for client-side session state: the loaded report, detected equations, the user's glossary, equation favourites, Ask-Anything conversation history, sidebar widths, and recent reports. Chosen over a server-side session because it requires no authentication and keeps the MVP single-user.

---

5. Implementation Methodology

5.1 Code organisation

The repository was structured to keep cross-cutting concerns isolated from feature code:

```
backend/app/
  routes/      # HTTP surface (upload.py, ai_tools.py)
  services/    # Domain logic (chatgpt_service.py, pdf_parser.py,
unit_converter.py)
  models/      # Pydantic request/response schemas
  utils/
  config.py    # Env-driven Settings
  main.py      # FastAPI app + lifespan cleanup task

frontend/src/
  components/  # UI components (one folder per concern)
  components/tools/  # Right-panel feature tools
  services/    # api.ts (Axios client, AbortSignal, clampContext)
  utils/       # errors, storage, toast, charToPage  (shared abstractions)
  types.ts     # Shared TypeScript contracts
```

```
  styles/      # globals.css (focus-visible, skeleton, drop-overlay,
toast)
```

5.2 Shared abstractions

A deliberate decision was made early on to factor out cross-cutting concerns into a small set of reusable utilities **before** scaling features. The benefit is that every tool — Summary, Explanation, Glossary, Evidence, Equation, Unit, ISA, Ask-Anything — interacts with the API, errors, loading, and copy-to-clipboard behaviour through the *same* primitives.

Abstraction	Concern
`frontend/src/types.ts`	Shared data contracts (`ReportData`, `EquationItem`, `RecentReport`).
`frontend/src/utils/errors.ts`	`describeApiError` — converts any Axios/network error into one human sentence (timeout, 413, 5xx, network, cancel).
`frontend/src/utils/storage.ts`	`readJSON`/`writeJSON`/`StorageKeys` — fault-tolerant `localStorage`.
`frontend/src/utils/toast.tsx`	`ToastProvider` + `useToast()` — global, non-blocking notifications.
`frontend/src/utils/charToPage.ts`	`charIndexToPage` — maps a character offset in the extracted text back to a PDF page.
`frontend/src/components/CopyButton.tsx`	One reusable copy control with toast feedback.
`frontend/src/components/KeyboardShortcuts.tsx`	Global help overlay opened with `?`.
`frontend/src/services/api.ts → clampContext`	Bounds the report text sent to the model so request size never explodes.

This is the most important methodological choice in the project: every feature is *thin* because the platform underneath is opinionated.

5.3 Per-feature implementation pattern

Every AI-backed tool follows the same 6-step pattern:

1. Validate input (early-return when there is nothing to send).
2. Cancel any in-flight request via `AbortController` and create a new one.
3. Show a skeleton/shimmer loading state.
4. Call the API helper with the abort signal (`reportService.<tool>(…, { signal })`).
5. On success → render result + `<CopyButton />`.
6. On failure → render `describeApiError(err)` in the same place; on cancel → silently exit.

This uniformity means a marker can read one tool and know how *every* tool behaves. It also makes accessibility uniform: all tools are keyboard-driven and announce loading/error states.

5.4 Equation detection pipeline

Equation detection is a hybrid pipeline because PDFs vary wildly:

1. **Server-side first.** The frontend calls `/api/detect-equations`, which the backend can implement against the OpenAI API or a deterministic detector.
2. **Client-side fallback.** If the server response fails or returns nothing, the frontend runs four regex families (LaTeX `$…$`, `\(…\)`, numeric `var = …`, and symbolic `[∫∑∏√±≈≤≥∞∂∇]`) and de-duplicates by case-folded equation text. Variable names are extracted with a stop-word filter.
3. **Rendering.** Every equation is rendered with KaTeX in the UI; favourites are persisted to `localStorage`.

5.5 Evidence Tracker

Implemented as text search rather than vector similarity, on purpose: the goal is to answer *"where in this PDF does that exact claim appear?"* not *"what is semantically similar?"*. The approach is:

1. Try an exact match of the highlighted selection in the extracted text.
2. Fall back to a case-insensitive match.
3. Map the matched character offset to a page number with `charIndexToPage`.
4. Return a 1000-character window of context around the match plus a type guess (Table / Figure / Equation / Text Reference).

The not-found case is reported back to the user in plain language ("Try selecting a shorter, unique phrase"), not silently.

5.6 Ask-Anything Q&A

Implemented as a per-report, persisted conversation:

- Each report ID maps to its own message history in `localStorage` (capped at the last 20 messages to bound storage).
- The report text is **clamped** before being sent to the model (`clampContext` keeps the head and tail of the document and elides the middle with a marker), so long PDFs never exceed the context window or blow up token bills.
- Requests are cancellable via `AbortController` (Cancel button doubles as the Send button while loading).

5.7 Robustness methodology

A deliberate effort was made to align the *whole* upload pipeline against a single time budget so failures are never just "axios timeout":

Layer	Setting	Value
Browser	Axios `timeout` (upload)	5 min
Nginx	`client_max_body_size`	60 MB

Layer	Setting	Value
Nginx	`client_body_timeout`	5 min
Nginx	`proxy_read_timeout`	5 min
Backend	FastAPI body size guard	50 MB
Frontend	Pre-flight extension + size check	.pdf, ≤ 50 MB

Errors at any layer are mapped to the same user-facing message style by `describeApiError`.

5.8 Resource hygiene

`backend/app/main.py` registers a FastAPI **lifespan** task that wakes up every `CLEANUP_INTERVAL_MINUTES` (default 60), scans `uploads/`, deletes any PDF whose mtime is older than `UPLOAD_RETENTION_HOURS` (default 24), and evicts the matching entry from `reports_store`. Both knobs are env-driven so deployment can tune them without code changes.

5.9 Accessibility methodology

Accessibility was treated as a cross-cutting requirement, not a polish step:

- `:focus-visible` rings (mouse users do not see them; keyboard users do).
- `aria-label`, `aria-pressed`, `aria-expanded`, `aria-controls`, `role="separator"`/`"tab"`/`"button"` on the relevant controls.
- A single `<h1>` per page; tools use `<h2>`/`<h3>` correctly.
- Global keyboard shortcuts: `Ctrl/Cmd + F` (search), `?` (help overlay), `Escape` (close).
- Skeleton/shimmer loading states announce themselves with `aria-busy`.

5.10 Version control and reproducibility

- Git on GitHub, with descriptive commit messages and an iterative history that mirrors the methodology cycles described in §1.
- A reproducible build via Docker Compose: a peer can clone the repo, set `OPENAI_API_KEY` in `backend/.env`, and run `docker compose up --build`.
- A smoke gate of `tsc --noEmit` and `react-scripts build` was used before large commits to keep `main` green.

---

6. Evaluation Methodology

Evaluation was performed continuously alongside development and in three modes:

1. **Functional manual testing** — exercising each feature against a small corpus of representative PDFs (a short native-text engineering report, a longer multi-section report, and a non-PDF file used to test the rejection path).
2. **Failure-mode injection** — uploading a > 50 MB file (expect 413), uploading a non-PDF (expect 400), stopping the backend mid-request (expect a network error message, not a crash), submitting an empty selection (expect disabled buttons, not a runtime error).

3. **Peer testing** — a peer attempted to use the deployed instance over a different network. This produced one of the most valuable findings of the project: the original 30-second Axios timeout was insufficient for large uploads on slower links, which led to the 5-minute budget aligned end-to-end.

Build-level verification:

- `tsc --noEmit` — strict TypeScript compile passes.
- `react-scripts build` (with `CI=true`) — production build succeeds without warnings.
- Backend syntax check via `python -m py_compile`.

Results are reported in the Evaluation chapter.

---

7. Ethical, Legal, and Security Considerations

- **API key handling.** `OPENAI_API_KEY` is loaded from `backend/.env`; the file is `.gitignore`d and never committed. The README documents the key rotation procedure (revoke at the OpenAI dashboard, regenerate, restart the backend).
- **User data.** Uploaded PDFs are stored only on disk in `uploads/` and are automatically pruned by the cleanup task. No user account, no analytics, no third-party tracking on the frontend.
- **Third-party dependency.** The system is functionally dependent on the OpenAI API. This is acknowledged as both a strength (best-in-class reasoning without GPU costs) and a risk (vendor lock-in, network dependency, cost exposure) — mitigated by making the model name an environment variable so it can be swapped for any OpenAI-compatible endpoint.
- **Licensing.** All dependencies are permissively licensed (MIT/BSD/Apache).

---

8. Limitations of the Methodology

These are recorded so they can be revisited in future work:

- **In-memory `reports_store`.** Restarting the backend loses parsed report metadata; PDFs on disk are still served, but the text/equations would need re-extraction. A persistent store (SQLite or Postgres) is the next step.
- **Text-only PDF parsing.** Scanned/image-only PDFs are not supported. Adding an OCR fallback (Tesseract) is straightforward and is listed as future work.
- **Single-user scope.** No authentication, no per-user isolation. A conscious choice to keep the MVP focused on the reading experience.
- **Cost coupling to OpenAI.** Each request costs money; aggressive use could exhaust a free tier. Context clamping mitigates this but does not eliminate it.
- **Manual evaluation only.** No automated end-to-end test suite was written for the MVP; this is a known gap and is recommended as immediate follow-up work (e.g. Playwright for the SPA, pytest for the FastAPI routes).

---

9. Summary

The methodology for *Read & Access* combines an iterative, user-feedback- driven cycle with deliberate up-front investment in shared abstractions — typed data contracts, centralised error mapping, persistence helpers, a toast system, and an `AbortController` request pattern — so that each individual feature could

remain thin and uniform. The architecture (React SPA → Nginx → FastAPI → OpenAI/filesystem) was chosen for the separation of concerns it provides and the ease of containerised reproducibility. Robustness, accessibility, and resource hygiene were treated as first-class methodological concerns rather than a finishing polish, and every design choice is traceable back to a stated functional or non-functional requirement.